Machine Learning: A Comprehensive Textbook

Solutions to Chapter 9 Exercises

Neural Networks: From Perceptron to Deep MLP

Solutions Manual

Exercise 9.1 — Backpropagation from Scratch

For a 2-hidden-layer network with sigmoid activations and MSE loss, derive $\nabla_{W^{[1]}} L, \nabla_{W^{[2]}} L, \nabla_{W^{[3]}} L$ step by step.

Solution. Network: $z^{[1]} = W^{[1]}x + b^{[1]}$, $a^{[1]} = \sigma(z^{[1]})$; $z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$, $a^{[2]} = \sigma(z^{[2]})$; $z^{[3]} = W^{[3]}a^{[2]} + b^{[3]}$, $\hat{y} = a^{[3]} = \sigma(z^{[3]})$ (or linear output, but we use sigmoid throughout as stated); loss $L = \frac{1}{2}\|\hat{y} - y\|^2$.

Output layer ($W^{[3]}$). Define the output error signal:

$$\delta^{[3]} = \frac{\partial L}{\partial z^{[3]}} = \frac{\partial L}{\partial a^{[3]}} \odot \frac{\partial a^{[3]}}{\partial z^{[3]}} = (\hat{y} - y) \odot \sigma'(z^{[3]}),$$

using $\partial L/\partial a^{[3]} = (\hat{y} - y)$ (from MSE) and $\partial a^{[3]}/\partial z^{[3]} = \sigma'(z^{[3]})$ elementwise. Since $z^{[3]} = W^{[3]}a^{[2]} + b^{[3]}$ is linear in $W^{[3]}$:

$$\nabla_{W^{[3]}} L = \delta^{[3]} \, (a^{[2]})^\top.$$

Hidden layer 2 ($W^{[2]}$). Backpropagate the error through $W^{[3]}$ to layer 2:

$$\delta^{[2]} = \frac{\partial L}{\partial z^{[2]}} = \left(\frac{\partial z^{[3]}}{\partial a^{[2]}}\right)^\top \delta^{[3]} \odot \sigma'(z^{[2]}) = \left((W^{[3]})^\top \delta^{[3]}\right) \odot \sigma'(z^{[2]}),$$

using $\partial z^{[3]}/\partial a^{[2]} = W^{[3]}$ (so its transpose maps the layer-3 error back to layer 2). Then, analogously,

$$\nabla_{W^{[2]}} L = \delta^{[2]} \, (a^{[1]})^\top.$$

Hidden layer 1 ($W^{[1]}$). Backpropagate again through $W^{[2]}$:

$$\delta^{[1]} = \left((W^{[2]})^\top \delta^{[2]}\right) \odot \sigma'(z^{[1]}),$$

and

$$\nabla_{W^{[1]}} L = \delta^{[1]} \, x^\top.$$

Summary (the backpropagation recursion):

$$\delta^{[3]} = (\hat{y} - y) \odot \sigma'(z^{[3]}), \qquad \delta^{[l]} = \left((W^{[l+1]})^\top \delta^{[l+1]}\right) \odot \sigma'(z^{[l]}) \;\; (l = 2, 1),$$

$$\nabla_{W^{[l]}} L = \delta^{[l]} (a^{[l-1]})^\top \quad (\text{with } a^{[0]} := x), \qquad \nabla_{b^{[l]}} L = \delta^{[l]}.$$

Each $\delta^{[l]}$ is computed from $\delta^{[l+1]}$ by (i) mapping backward through the transpose of the forward weight matrix, and (ii) multiplying elementwise by the local activation derivative $\sigma'(z^{[l]}) = a^{[l]} \odot (1 - a^{[l]})$ (Exercise 5.1a) — this backward recursive structure, computing each layer's gradient from the next layer's, is exactly what makes backpropagation an O(network size) algorithm rather than requiring a separate, independent computation for every weight.

Exercise 9.2 — Softmax + Cross-Entropy Gradient

Prove that for softmax output and cross-entropy loss, $\delta_k^{[L+1]} = \hat{p}_k - y_k$.

Solution. Let $z = z^{[L+1]} \in \mathbb{R}^K$ be the logits, $\hat{p}_k = \text{softmax}(z)_k = \dfrac{e^{z_k}}{\sum_j e^{z_j}}$, and the loss for one-hot label y be $L = -\sum_k y_k \log \hat{p}_k$. We want $\delta_k := \partial L / \partial z_k$.

Step 1: Jacobian of softmax. Compute $\partial \hat{p}_i / \partial z_k$ for all i, k. Write $\hat{p}_i = e^{z_i}/S$ where $S = \sum_j e^{z_j}$.

$$\frac{\partial \hat{p}_i}{\partial z_k} = \frac{\partial}{\partial z_k}\left(\frac{e^{z_i}}{S}\right) = \frac{\mathbf{1}[i=k] e^{z_i} S - e^{z_i} e^{z_k}}{S^2} = \mathbf{1}[i=k]\hat{p}_i - \hat{p}_i \hat{p}_k = \hat{p}_i(\mathbf{1}[i=k] - \hat{p}_k).$$

Step 2: Chain rule through cross-entropy.

$$\delta_k = \frac{\partial L}{\partial z_k} = \sum_i \frac{\partial L}{\partial \hat{p}_i} \cdot \frac{\partial \hat{p}_i}{\partial z_k} = \sum_i \left(-\frac{y_i}{\hat{p}_i}\right) \cdot \hat{p}_i(\mathbf{1}[i=k] - \hat{p}_k) = -\sum_i y_i(\mathbf{1}[i=k] - \hat{p}_k).$$

Distribute:

$$= -\sum_i y_i \mathbf{1}[i=k] + \hat{p}_k \sum_i y_i = -y_k + \hat{p}_k \cdot 1$$

(using $y_k = \sum_i y_i \mathbf{1}[i=k]$ by definition, and $\sum_i y_i = 1$ since y is a one-hot/valid probability vector). Hence

$$\delta_k = \hat{p}_k - y_k. \qquad \blacksquare$$

This remarkably simple result — the gradient of the combined softmax+cross-entropy loss with respect to the logits is just "prediction minus target" — is exactly analogous to the binary logistic regression gradient (Exercise 5.2), and is precisely why frameworks implement a fused, numerically-stable `softmax_cross_entropy` operation rather than computing the softmax Jacobian and cross-entropy gradient separately.

Exercise 9.3 — He Initialisation Derivation

Prove $\text{Var}[\text{ReLU}(z)] = \frac{1}{2}\text{Var}[z]$ for $z \sim \mathcal{N}(0, \sigma^2)$. Derive $\sigma_w^2 = 2/n_{l-1}$.

Solution. **Variance of ReLU output.** Let $z \sim \mathcal{N}(0, \sigma^2)$, symmetric about 0, so $P(z > 0) = P(z < 0) = 1/2$. $\text{ReLU}(z) = \max(0, z)$. Since $\mathbb{E}[z] = 0$ and z is symmetric, $\mathbb{E}[z \mid z > 0]$ relates to the half-normal distribution; we compute directly:

$$\mathbb{E}[\text{ReLU}(z)^2] = \mathbb{E}[\max(0,z)^2] = \int_0^\infty z^2 f_z(z)\, dz,$$

where f_z is the $\mathcal{N}(0, \sigma^2)$ density. By symmetry of $z^2 f_z(z)$ (an even function, since both z^2 and f_z restricted appropriately are even) around 0:

$$\int_0^\infty z^2 f_z(z)\, dz = \frac{1}{2}\int_{-\infty}^\infty z^2 f_z(z)\, dz = \frac{1}{2}\mathbb{E}[z^2] = \frac{1}{2}\sigma^2$$

(using $\mathbb{E}[z^2] = \text{Var}(z) = \sigma^2$ since $\mathbb{E}[z] = 0$). Also, $\mathbb{E}[\text{ReLU}(z)] = \int_0^\infty z f_z(z)\, dz = \frac{\sigma}{\sqrt{2\pi}}$ (a standard half-normal mean calculation), which is nonzero, but for the purposes of the He-initialization heuries we track second-moment matching for pre-activation propagation, treating $\text{Var}[\text{ReLU}(z)] \approx$

$\mathbb{E}[\text{ReLU}(z)^2]$ (the standard derivation in He et al. 2015 uses $\mathbb{E}[\text{ReLU}(z)^2]$ directly, effectively assuming the small mean-squared term is negligible relative to the goal of matching forward-pass variance across layers — the key quantity needed is exactly $\mathbb{E}[\text{ReLU}(z)^2] = \frac{1}{2}\sigma^2 = \frac{1}{2}\text{Var}[z]$):

$$\text{Var}[\text{ReLU}(z)] \approx \mathbb{E}[\text{ReLU}(z)^2] = \frac{1}{2}\text{Var}[z]. \qquad \blacksquare$$

Deriving He initialization. Consider a layer with n_{l-1} inputs $a_1, \ldots, a_{n_{l-1}}$ (post-ReLU activations from the previous layer, each with variance $v_{l-1} := \text{Var}[a_{l-1}]$ and, by design, mean ≈ 0 after appropriate centering assumptions or more precisely by the standard He-init argument treating pre-activations as zero-mean), and weights $w_j \sim \mathcal{N}(0, \sigma_w^2)$ i.i.d., independent of the inputs. The pre-activation of the next layer is $z = \sum_{j=1}^{n_{l-1}} w_j a_j$. Assuming weights and activations are independent with zero-mean weights:

$$\text{Var}[z] = \sum_{j=1}^{n_{l-1}} \text{Var}[w_j a_j] = n_{l-1} \cdot \text{Var}[w] \cdot \mathbb{E}[a^2] = n_{l-1}\sigma_w^2 \cdot \mathbb{E}[a^2]$$

(using $\text{Var}[w_j a_j] = \mathbb{E}[w_j^2]\mathbb{E}[a_j^2] - (\mathbb{E}[w_j]\mathbb{E}[a_j])^2 = \sigma_w^2 \mathbb{E}[a_j^2]$ since $\mathbb{E}[w_j] = 0$, and $\mathbb{E}[a_j^2]$ is the previous layer's post-ReLU second moment, which by the result above equals $\mathbb{E}[a^2] = \frac{1}{2}\text{Var}[z_{l-1}]$ where z_{l-1} was the previous layer's pre-activation).

To **preserve variance across layers** (avoiding exponential vanishing/exploding of activations through depth), we want $\text{Var}[z_l] = \text{Var}[z_{l-1}]$, i.e. $n_{l-1}\sigma_w^2 \cdot \frac{1}{2}\text{Var}[z_{l-1}] = \text{Var}[z_{l-1}]$, which requires

$$n_{l-1}\sigma_w^2 \cdot \frac{1}{2} = 1 \implies \sigma_w^2 = \frac{2}{n_{l-1}}. \qquad \blacksquare$$

This is the **He initialization** variance: weights should be initialized $\sim \mathcal{N}(0, 2/n_{l-1})$ (or the corresponding uniform variant), with the factor of 2 (compared to Xavier/Glorot's factor of 1 used for tanh/sigmoid) precisely compensating for ReLU zeroing out (on average) half of its inputs' variance contribution, as derived above.

Exercise 9.4 — Adam Bias Correction

Show that if $m_0 = 0$ and gradient is constant $\hat{g}_t = g$, then $\mathbb{E}[m_t] = (1 - \beta_1^t)g$, and $\hat{m}_t = m_t/(1 - \beta_1^t)$ is unbiased.

Solution. Adam's first-moment (momentum) update is $m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$, with $m_0 = 0$. Suppose the gradient is constant, $g_t = g$ for all t (a simplification to derive the bias-correction formula exactly, as in the original Adam paper). Unroll the recursion:

$$m_1 = \beta_1 \cdot 0 + (1 - \beta_1)g = (1 - \beta_1)g,$$

$$m_2 = \beta_1 m_1 + (1 - \beta_1)g = \beta_1(1 - \beta_1)g + (1 - \beta_1)g = (1 - \beta_1)g(1 + \beta_1),$$

$$m_3 = \beta_1 m_2 + (1 - \beta_1)g = (1 - \beta_1)g(1 + \beta_1 + \beta_1^2),$$

and by induction,

$$m_t = (1 - \beta_1)g \sum_{i=0}^{t-1} \beta_1^i = (1 - \beta_1)g \cdot \frac{1 - \beta_1^t}{1 - \beta_1} = (1 - \beta_1^t)\, g$$

(using the geometric series sum $\sum_{i=0}^{t-1} \beta_1^i = \frac{1-\beta_1^t}{1-\beta_1}$ for $\beta_1 \neq 1$). Since g is treated as deterministic/constant here (or, in the more general stochastic-gradient version of this argument, we'd take $\mathbb{E}[g_t] = g$ for all t and the identical unrolling applies to expectations directly since the recursion is linear):

$$\mathbb{E}[m_t] = (1-\beta_1^t)\, g \quad (\text{or } = (1-\beta_1^t)\, \mathbb{E}[g_t] \text{ in the general i.i.d.-gradient case}). \qquad \blacksquare$$

Bias correction. Notice $\mathbb{E}[m_t] = (1-\beta_1^t)\, g \neq g$ for finite t — m_t is a *biased* estimate of the true gradient g, systematically too small in magnitude especially for small t (since β_1 close to 1, e.g. $\beta_1 = 0.9$, means β_1^t decays slowly, so early iterations have $1-\beta_1^t$ far from 1 — e.g. at $t=1$, $1-\beta_1 = 0.1$, meaning m_1 underestimates g by a factor of 10). Define the bias-corrected estimate:

$$\hat{m}_t := \frac{m_t}{1-\beta_1^t}.$$

Then

$$\mathbb{E}[\hat{m}_t] = \frac{\mathbb{E}[m_t]}{1-\beta_1^t} = \frac{(1-\beta_1^t)g}{1-\beta_1^t} = g,$$

exactly unbiased for any $t \geq 1$ (as long as $\beta_1^t \neq 1$, which holds for $\beta_1 < 1$). $\blacksquare$ This is exactly the bias-correction step used in the Adam optimizer, necessary because initializing $m_0 = 0$ (and similarly $v_0 = 0$ for the second-moment estimate) creates a systematic bias toward zero in early iterations that must be corrected, especially important when β_1, β_2 are close to 1 (their typical default values, e.g. $\beta_1 = 0.9, \beta_2 = 0.999$).

Exercise 9.5 — Dropout as ℓ_2 Regularisation

For $\hat{y} = w^\top \tilde{x}$ with dropout $\tilde{x}_j = m_j x_j/(1-p)$, compute $\mathbb{E}_m[(\hat{y}-y)^2]$ and show it equals $(\bar{y}-y)^2 + \frac{p}{1-p}\sum_j w_j^2 x_j^2 + O(p^2)$.

Solution. Here $m_j \sim \text{Bernoulli}(1-p)$ i.i.d. (each input kept with probability $1-p$, dropped with probability p), and $\tilde{x}_j = m_j x_j/(1-p)$ (inverted dropout scaling, ensuring $\mathbb{E}_m[\tilde{x}_j] = x_j$). The prediction is $\hat{y} = w^\top \tilde{x} = \sum_j w_j \tilde{x}_j = \sum_j w_j m_j x_j/(1-p)$.

Mean of $\hat{y}$. $\mathbb{E}_m[\hat{y}] = \sum_j w_j x_j \mathbb{E}[m_j]/(1-p) = \sum_j w_j x_j \cdot (1-p)/(1-p) = \sum_j w_j x_j =: \bar{y}$ (the "clean" prediction with no dropout, matching the notation in the problem).

Variance of $\hat{y}$. Since the m_j are independent across j,

$$\text{Var}_m(\hat{y}) = \sum_j w_j^2 x_j^2 \,\text{Var}(m_j)/(1-p)^2.$$

For $m_j \sim \text{Bernoulli}(1-p)$: $\text{Var}(m_j) = (1-p)\cdot p$ (standard Bernoulli variance = prob. of success $\times$ prob. of failure). So

$$\text{Var}_m(\hat{y}) = \sum_j w_j^2 x_j^2 \cdot \frac{p(1-p)}{(1-p)^2} = \frac{p}{1-p}\sum_j w_j^2 x_j^2.$$

Expected squared error. Using the bias-variance-style decomposition $\mathbb{E}_m[(\hat{y}-y)^2] = \text{Var}_m(\hat{y}) + (\mathbb{E}_m[\hat{y}]-y)^2$ (standard identity: $\mathbb{E}[(\hat{y}-y)^2] = \mathbb{E}[(\hat{y}-\mathbb{E}\hat{y})^2] + (\mathbb{E}\hat{y}-y)^2$, since y is a fixed constant w.r.t. the dropout randomness m):

$$\mathbb{E}_m[(\hat{y}-y)^2] = (\bar{y}-y)^2 + \frac{p}{1-p}\sum_j w_j^2 x_j^2.$$

This is already the *exact* result (no approximation needed at this stage — the calculation above is exact for any $p \in [0, 1)$). The $O(p^2)$ notation in the problem statement reflects that when this exact expression $\frac{p}{1-p}$ is itself Taylor-expanded for small p, $\frac{p}{1-p} = p + p^2 + p^3 + \cdots = p + O(p^2)$, so to leading order in small p:

$$\mathbb{E}_m[(\hat{y} - y)^2] \approx (\bar{y} - y)^2 + p \sum_j w_j^2 x_j^2 + O(p^2). \qquad \blacksquare$$

Interpretation. The expected dropout loss equals the ordinary (clean) squared-error loss *plus* an extra term $\frac{p}{1-p} \sum_j w_j^2 x_j^2$ that penalizes large weights w_j scaled by the corresponding input magnitude x_j^2 — this is exactly an (input-dependent, adaptive) ℓ_2 (Ridge-style) regularization penalty on the weights, with effective regularization strength controlled by the dropout probability p (larger $p \Rightarrow$ stronger implicit ℓ_2 penalty). This formally demonstrates the well-known heuristic that dropout acts as an adaptive form of ℓ_2/weight-decay regularization, though — importantly — the penalty here is *input-dependent* (x_j^2-weighted) rather than a fixed uniform penalty like standard Ridge, and (in a full nonlinear network) also interacts with the specific data distribution at each layer, which is why dropout's regularizing effect is richer than plain weight decay in practice.

Exercise 9.6 — Vanishing Gradients

For a 20-layer sigmoid network, estimate $\partial L/\partial W^{[1]}$ relative to $\partial L/\partial W^{[20]}$. Compare for ReLU.

Solution. From the backpropagation recursion (Exercise 9.1), each layer's error signal picks up a multiplicative factor of $(W^{[l+1]})^\top$ and $\sigma'(z^{[l]})$:

$$\delta^{[l]} = \left((W^{[l+1]})^\top \delta^{[l+1]}\right) \odot \sigma'(z^{[l]}).$$

So $\delta^{[1]}$ (and hence $\nabla_{W^{[1]}} L = \delta^{[1]} x^\top$) involves a product of *19* such factors chained back from layer 20 to layer 1, each contributing a term of order $\|W^{[l]}\| \cdot \max \sigma'(z)$.

Sigmoid case. The derivative $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ has *maximum value* $1/4$, attained only at $z = 0$; for typical (nonzero) pre-activations, $\sigma'(z)$ is usually meaningfully smaller than this. Even taking the most optimistic case where every layer operates exactly at the peak $\sigma'(z) = 1/4$ and weight matrices have spectral norm ≈ 1 (an idealized, best-case scenario — in practice weights are often initialized with smaller norms specifically to control this), the gradient magnitude ratio scales as roughly

$$\frac{\|\nabla_{W^{[1]}} L\|}{\|\nabla_{W^{[20]}} L\|} \sim \left(\frac{1}{4}\right)^{19} \approx 1.1 \times 10^{-12}$$

(19 layers of backpropagated sigmoid-derivative attenuation between layer 20 and layer 1). In realistic settings where $\sigma'(z)$ frequently takes much smaller values than its peak (since most pre-activations are not exactly at $z = 0$) and/or weight matrix norms are < 1 compounding further, the true attenuation is typically far more severe than this already-tiny idealized bound — this quantitatively demonstrates the **vanishing gradient problem**: gradients reaching the earliest layers of a very deep sigmoid network are essentially zero in practice, so early layers receive almost no learning signal and train extremely slowly if at all.

ReLU case. $\mathrm{ReLU}'(z) = \mathbf{1}[z > 0] \in \{0, 1\}$ — for any *active* unit ($z > 0$), the local derivative is exactly 1 (no attenuation at all), not a fraction < 1 as with sigmoid. So for a path through the network where all units along the path are active, the gradient magnitude ratio is governed *purely* by the products of weight-matrix norms $\prod_l \|W^{[l]}\|$, with *no* additional multiplicative shrinkage from the

activation derivative itself (unlike sigmoid's guaranteed $\leq 1/4$ factor at every single layer). With well-scaled (e.g. He-initialized, Exercise 9.3) weights designed so $\|W^{[l]}\| \approx 1$ in the appropriate sense, gradients can propagate through many more layers without vanishing — this is precisely why ReLU (and its variants) enabled training substantially deeper networks than were practical with sigmoid/tanh activations, and is one of the key reasons for ReLU's widespread adoption in modern deep learning, alongside architectural innovations like residual connections (Exercise 10.4) that provide an even more direct gradient path.

Exercise 9.7 — UAT Limitations

(a) State precisely what the UAT guarantees. (b) Construct a function needing exponentially many neurons in one hidden layer but efficiently representable with depth. (c) Why prefer depth over width?

Solution. **(a) Precise statement.** The Universal Approximation Theorem (Cybenko 1989, Hornik 1991) states: for any continuous function $f : K \to \mathbb{R}$ on a compact set $K \subset \mathbb{R}^d$, and any $\epsilon > 0$, there exists a single-hidden-layer network $g(x) = \sum_{i=1}^{N} c_i \, \sigma(w_i^\top x + b_i)$ (with a non-polynomial, e.g. sigmoidal, activation σ) and *some sufficiently large* number of hidden units N, such that $\sup_{x \in K} |f(x) - g(x)| < \epsilon$. Crucially, the theorem is a pure **existence** statement — it guarantees such a network exists for *any* target function and *any* desired accuracy, but it says *nothing* about how large N needs to be (it can be astronomically, even exponentially, large as a function of input dimension d and desired accuracy ϵ), and it says nothing about whether such weights are *learnable* by gradient descent from a practical amount of training data in a practical amount of time.

(b) A function requiring exponentially many single-layer units. A canonical example is computing the **parity function** (or more generally certain highly-oscillatory / compositional functions) on d binary inputs: $f(x_1, \ldots, x_d) = x_1 \oplus x_2 \oplus \cdots \oplus x_d$ (XOR of d bits). It is a well-known circuit-complexity result that parity requires a depth-2 circuit (equivalently, a single-hidden-layer threshold/sigmoid network) of *exponential* size ($\Omega(2^d)$ gates/units in the worst analyses for bounded-depth circuits, per classical results such as Håstad's switching lemma for AC^0 circuits) to compute exactly, because a shallow network essentially must enumerate exponentially many "regions"/parity-sensitive combinations of the input bits with a single layer of nonlinearity. In sharp contrast, parity is trivially computable with a *deep* network of $O(d)$ total units: build a chain of $d-1$ XOR gates, each combining the running XOR with the next input bit, requiring only $O(1)$ units per layer and $O(d)$ layers — i.e. linear (not exponential) total size when depth is allowed to grow. (Similar exponential-separation results are known more generally for certain classes of compositional/hierarchical functions, formalized rigorously in results such as Telgarsky's 2016 depth-separation theorems for ReLU networks, and Eldan & Shamir's 2016 depth-separation result for a specific radially-symmetric function that requires exponentially many units to approximate with 2 layers but is efficiently representable with 3 layers.)

(c) Why prefer depth over width. This exponential gap illustrates that width alone (a single hidden layer, as covered by the classical UAT) can require exponentially more parameters/units than a network that is also allowed to be deep, for the *same* target function class (specifically, for functions that are naturally compositional/hierarchical, which includes essentially all functions of practical interest in vision, language, etc., which decompose into layers of increasingly abstract features — edges → textures → parts → objects, in the case of images). Depth allows the network to reuse and compose intermediate computations across layers (each layer builds on the previous layer's representations), achieving exponentially more *expressive efficiency* per parameter than a

shallow-but-wide network, which must instead represent each distinct combination of input features somewhat independently within its single layer of nonlinearity. This is the fundamental theoretical justification (complementing the practical/empirical evidence) for why practitioners favor deep architectures over shallow-but-arbitrarily-wide ones — depth is not merely a matter of taste or historical convention, but provably yields exponentially more compact representations for an important, broad class of compositional target functions.

Exercise 9.8 — Programming: MLP from Scratch

Implement an MLP from scratch in NumPy, with forward/backward pass, MSE and cross-entropy loss, SGD with momentum, He init, verified via numerical gradient checking.

Solution. **Reference implementation.**

```
import numpy as np

def sigmoid(z): return 1/(1+np.exp(-z))
def sigmoid_grad(z): s = sigmoid(z); return s*(1-s)
def relu(z): return np.maximum(0, z)
def relu_grad(z): return (z > 0).astype(float)

class MLP:
    def __init__(self, sizes, activation='relu'):
        self.L = len(sizes) - 1
        self.act, self.act_grad = (relu, relu_grad) if activation=='relu' \
                                  else (sigmoid, sigmoid_grad)
        self.W, self.b = {}, {}
        for l in range(1, self.L+1):
            fan_in = sizes[l-1]
            # He initialisation (Exercise 9.3): sigma_w^2 = 2/fan_in
            self.W[l] = np.random.randn(sizes[l], fan_in) * np.sqrt(2.0/fan_in)
            self.b[l] = np.zeros((sizes[l], 1))
        self.vW = {l: np.zeros_like(self.W[l]) for l in self.W}  # momentum buffers
        self.vb = {l: np.zeros_like(self.b[l]) for l in self.b}

    def forward(self, X):                       # X: (n_features, n_samples)
        a, z = {0: X}, {}
        for l in range(1, self.L+1):
            z[l] = self.W[l] @ a[l-1] + self.b[l]
            a[l] = self.act(z[l]) if l < self.L else z[l]   # linear output layer
        self.cache = (a, z)
        return a[self.L]

    def backward(self, yhat, y, loss='mse'):
        a, z = self.cache
        n = y.shape[1]
        grads_W, grads_b = {}, {}
        # output error (Exercise 9.1/9.2)
        if loss == 'mse':
```

```
            delta = (yhat - y) / n
        elif loss == 'softmax_ce':               # delta = phat - y  (Exercise 9.2)
            delta = (yhat - y) / n
        for l in range(self.L, 0, -1):
            grads_W[l] = delta @ a[l-1].T
            grads_b[l] = delta.sum(axis=1, keepdims=True)
            if l > 1:
                delta = (self.W[l].T @ delta) * self.act_grad(z[l-1])
        return grads_W, grads_b

    def step(self, grads_W, grads_b, lr=0.01, momentum=0.9):
        for l in self.W:
            self.vW[l] = momentum*self.vW[l] - lr*grads_W[l]
            self.vb[l] = momentum*self.vb[l] - lr*grads_b[l]
            self.W[l] += self.vW[l]
            self.b[l] += self.vb[l]

def numerical_gradient_check(mlp, X, y, loss='mse', eps=1e-5):
    """Compare analytic backprop gradient to finite differences."""
    yhat = mlp.forward(X)
    grads_W, _ = mlp.backward(yhat, y, loss=loss)
    l = 1  # check first layer's weights
    W_orig = mlp.W[l].copy()
    max_rel_err = 0
    for idx in np.ndindex(W_orig.shape[:2]):  # sample a few entries
        mlp.W[l][idx] += eps
        loss_plus = 0.5*np.mean((mlp.forward(X)-y)**2) if loss=='mse' else None
        mlp.W[l][idx] -= 2*eps
        loss_minus = 0.5*np.mean((mlp.forward(X)-y)**2) if loss=='mse' else None
        mlp.W[l][idx] += eps  # restore
        num_grad = (loss_plus - loss_minus) / (2*eps)
        rel_err = abs(num_grad - grads_W[l][idx]) / (abs(num_grad)+abs(grads_W[l][idx])+1e-8)
        max_rel_err = max(max_rel_err, rel_err)
    return max_rel_err  # should be < 1e-5 for correct implementation
```

Verification methodology. Gradient checking compares the analytic backprop gradient $\partial L/\partial W_{ij}$ against the centered finite-difference approximation $\dfrac{L(W_{ij}+\epsilon) - L(W_{ij}-\epsilon)}{2\epsilon}$ for a small ϵ (typically 10^{-4} to 10^{-7}), for a sample of individual weight entries. A correct implementation should show relative error on the order of 10^{-7} or smaller (limited primarily by floating-point precision); relative errors above $\sim 10^{-4}$ typically indicate a bug in the backward pass (e.g. a transposed matrix, missing elementwise multiplication by the activation derivative, or an off-by-one in the layer indexing of the backprop recursion from Exercise 9.1). This numerical check is the standard, essential sanity check whenever implementing backpropagation manually (i.e. without an autograd framework), since a subtly wrong analytic gradient can still allow the network to *appear* to train (loss decreasing) while converging to a suboptimal or entirely incorrect solution.